Probabilistic Forecasting with Temporal Convolutional Neural Network

Yitian Chen[a], Yanfei Kang[b,*], Yixiong Chen[c], Zizhuo Wang[d]

[a]*Bigo Beijing R&D Center, Bigo Inc., Beijing 100191, China.*
[b]*School of Economics and Management, Beihang University, Beijing 100191, China.*
[c]*IBM China CIC, KIC Technology Center, Shanghai 200433, China.*
[d]*Institute for Data and Decision Analytics, The Chinese University of Hong Kong, Shenzhen, 518172, China.*

arXiv:1906.04397v3 [stat.ML] 16 Mar 2020

Abstract

We present a probabilistic forecasting framework based on convolutional neural network (CNN) for multiple related time series forecasting. The framework can be applied to estimate probability density under both parametric and non-parametric settings. More specifically, stacked residual blocks based on dilated causal convolutional nets are constructed to capture the temporal dependencies of the series. Combined with representation learning, our approach is able to learn complex patterns such as seasonality, holiday effects within and across series, and to leverage those patterns for more accurate forecasts, especially when historical data is sparse or unavailable. Extensive empirical studies are performed on several real-world datasets, including datasets from JD.com, China's largest online retailer. The results show that our framework compares favorably to the state-of-the-art in both point and probabilistic forecasting.

Keywords: Probabilistic forecasting, convolutional neural network, dilated causal convolution, demand forecasting, high-dimensional time series

1. Introduction

Time series forecasting plays a key role in many business decision-making scenarios, such as managing limited resources, optimizing operational processes, among others. Most existing forecasting methods focus on point forecasting, i.e., forecasting the conditional mean or median of future observations. However, probabilistic forecasting becomes increasingly important as it is able to extract richer information from historical data and better capture the uncertainty of the future. In retail business, probabilistic forecasting of product supply and demand is fundamental for successful procurement process and optimal inventory planning. Also, probabilistic shipment forecasting, i.e., generating probability distributions of the delivery volumes of packages, is the key component of the consequent logistics operations, such as labor resource planning and

*Corresponding author
Email addresses: chenyitian@bigo.sg (Yitian Chen), yanfeikang@buaa.edu.cn (Yanfei Kang), chenyixiong516@msn.com (Yixiong Chen), wangzizhuo@cuhk.edu.cn (Zizhuo Wang)

delivery vehicle deployment.

In such circumstances, instead of predicting individual or a small number of time series, one needs to predict thousands or millions of related series. Moreover, there are many more challenges in real-world applications. For instance, new products emerge weekly on retail platforms and one often needs to forecast the demand of products without historical shopping festival data (e.g., Black Friday in North America, "11.11" shopping festival in China). Furthermore, forecasting often requires the consideration of exogenous variables that have significant influence on future demand (e.g., promotion plans provided by operations teams, accurate weather forecasts for brick and mortar retailers). Such forecasting problems can be extended to a variety of domains. Examples include forecasting the web traffic for internet companies (Kaggle, 2017), the energy consumption for individual households, the load for servers in a data center (Salinas et al., 2019) and traffic flows in transportation domain (Lv et al., 2015).

Classical forecasting methods, such as ARIMA (AutoRegressive Integrated Moving Average, Box et al., 2015) and exponential smoothing (Hyndman et al., 2008), are widely employed for univariate base-level forecasting. To incorporate exogenous covariates, several extensions of these methods have been proposed, such as ARIMAX (AutoRegressive Integrated Moving Average with Explanatory Variable) and dynamic regression models (Hyndman and Athanasopoulos, 2018). These models are well-suited for applications in which the structure of the data is well understood and there is sufficient historical data. However, working with thousands or millions of series requires prohibitive labor and computing resources for parameter estimation. Moreover, they are not applicable in situations where historical data is sparse or unavailable.

Models based on Recurrent neural network (RNN) (Graves, 2013) and the sequence to sequence (Seq2Seq) framework (Cho et al., 2014; Sutskever et al., 2014) have achieved great success in many different sequential tasks such as machine translation (Sutskever et al., 2014), language modeling (Mikolov et al., 2010) and recently time series forecasting (Laptev et al., 2017; Wen et al., 2017; Salinas et al., 2019; Rangapuram et al., 2018; Sagheer and Kotb, 2019; Shen et al., 2019). For example, in the forecasting competition community, the Seq2Seq model based on a gated recurrent unit (GRU) (Cho et al., 2014) won the Kaggle web traffic forecasting competition (Suilin, 2017). A hybrid model that combines exponential smoothing method and RNN won the M4 forecasting competition, which consists of 100,000 series with different seasonal patterns (Makridakis et al., 2018*a*). However, training with back propagation through time (BPTT) algorithm often hampers efficient computation. In addition, training RNN can be remarkably difficult (Werbos, 1990; Pascanu et al., 2013). Dilated causal convolutional architectures, e.g., Wavenet (van den Oord et al., 2016), offers an alternative for modeling

sequential data. By stacking layers of dilated causal convolutional nets, receptive fields can be increased, and the long-term correlations can be captured without violating the temporal orders. In addition, in dilated causal convolutional architectures, the training process can be performed in parallel, which guarantees computation efficiency.

Most Seq2Seq frameworks or Wavenet (van den Oord et al., 2016) are autoregressive generative models that factorize the joint distribution as a product of the conditionals. In this setting, a one-step-ahead prediction approach is adopted, i.e., first a prediction is generated by using the past observations, and the generated result is then fed back as the ground truth to make further forecasts. More recent research shows that non-autoregressive approaches or direct prediction strategy, predicting observations of all time steps directly, can achieve better performances (Gu et al., 2017; Bai et al., 2018; Wen et al., 2017). In particular, non-autoregressive models are more robust to mis-specification by avoiding error accumulation and thus yield better prediction accuracy. Moreover, training over all the prediction horizons can be parallelized.

Having reviewing all these challenges and developments, in this paper, we propose the Deep Temporal Convolutional Network (DeepTCN), a non-autoregressive probabilistic forecasting framework for large collections of related time series. The main contributions of the paper are as follows:

- We propose a CNN-based forecasting framework that provides both parametric and non-parametric approaches for probability density estimation.
- The framework, being able to learn latent correlation among series and handle complex real-world forecasting situations such as data sparsity and cold starts, shows high scalability and extensibility.
- The model is very flexible and can include exogenous covariates such as additional promotion plans or weather forecasts.
- Extensive empirical studies show our framework compares favorably to state-of-the-art methods in both point forecasting and probabilistic forecasting tasks.

The rest of this paper is organized as follows. Section 2 provides a brief review of related work on time series forecasting and deep learning methods for forecasting. In Section 3, we describe the proposed forecasting method, including the neural network architectures, the probabilistic forecasting framework, and the input features. We demonstrate the superiority of the proposed approach via extensive experiments in Section 4 and conclude the paper in Section 5.

2. Related Work

Earlier studies on time series forecasting are mostly based on statistical models, which are mainly generative models based on state space framework such as exponential smoothing, ARIMA models and several other extensions. For these methods, Hyndman et al. (2008) and Box et al. (2015) provide a comprehensive overview in the context of univariate forecasting.

In recent years, large number of related series are emerging in the routine functioning of many companies. Not sharing information from other time series, traditional univariate forecasting methods fit a model for each individual time series, and thus cannot learn across similar time series. Moreover, numerous researchers have shown that pure machine learning methods could not outperform statistical approaches in forecasting individual time series, the reasons for which can be attributed to overfitting and non-stationarity (Bandara et al., 2020; Makridakis et al., 2018*b*). Therefore, methods that can provide forecasting on multiple series jointly have received increasing attention in the last few years (e.g., Yu et al., 2016).

Both RNNs and CNNs have been shown to be able to model complex nonlinear feature interactions and yield substantial forecasting performances, especially when many related time series are available (Smyl, 2016; Laptev et al., 2017; Wen et al., 2017; Salinas et al., 2019; Rangapuram et al., 2018). For example, Long Short-Term Memory (LSTM), one type of RNN architecture, won the CIF2016 forecasting competition for monthly time series (Stepnicka and Burda, 2016). Bianchi et al. (2017) compare a variety of RNNs in their performances in the Short Term Load Forecasting problem. Borovykh et al. (2017) investigate the application of CNNs to financial time series forecasting.

To better understand the uncertainty of the future, probabilistic forecasting with deep learning models has attracted increasing attention. DeepAR (Salinas et al., 2019), which trains an auto-regressive RNN model on a rich collection of similar time series, produces more accurate probabilistic forecasts on several real-world data sets. The deep state space models (DeepState), presented by Rangapuram et al. (2018), combine state space models with deep learning and can retain data efficiency and interpretability while learning the complex patterns from raw data. Under a similar scheme, Maddix et al. (2018) propose the combination of deep neural networks and Gaussian Process. More recently, Gasthaus et al. (2019) propose SQF-RNN, a probabilistic framework to model conditional quantile functions with isotonic splines, which allows more flexible output distributions.

Most of these probabilistic forecasting frameworks are autoregressive models, which use recursive strategy to generate multi-step forecasts. In neural machine translation, non-autoregressive translation (NAT) models have achieved significant speedup at the cost of slightly inferior ac-

curacy compared to autoregressive translation models (Gu et al., 2017). For example, Bai et al. (2018) propose a non-autoregressive framework based on dilated causal convolution and the empirical study on multiple datasets shows that the framework outperforms generic recurrent architectures such as LSTMs and GRUs. In forecasting applications, non-autoregressive approaches have also been shown to be less biased and more robust. Recently, Wen et al. (2017) present a multi-horizon quantile recurrent forecaster to combine sequential neural nets and quantile regression (Koenker and Bassett Jr, 1978). By training on all time points at the same time, their framework can significantly improve the training stability and the forecasting performances of recurrent nets.

Our method differs from the aforementioned approaches in the following ways. First, stacked dilated causal convolutional nets are constructed to represent the encoder and model the stochastic process of historical observations of series. Instead of applying gating mechanism (e.g., in Wavenet (van den Oord et al., 2016)), residual blocks are used for the dilated causal convolutional nets to extract information of historical observations and help achieve superior forecasting accuracy. Second, inspired by the dynamic regression models (Pankratz, 2012) such as ARIMAX, in the decoder part, a novel variant of the residual neural network is proposed to incorporate information from both past observations and exogenous covariates. Finally, our model enjoys the flexibility to embrace a variety of probability density estimation approaches.

3. Method

A general probabilistic forecasting problem for multiple related time series can be described as follows: Given a set of time series $\mathbf{y}_{1:t} = \{y_{1:t}^{(i)}\}_{i=1}^{N}$, we denote the future time series as $\mathbf{y}_{(t+1):(t+\Omega)} = \{y_{(t+1):(t+\Omega)}^{(i)}\}_{i=1}^{N}$, where N is the number of series, t is the length of the historical observations and Ω is the length of the forecasting horizon. Our goal is to model the conditional distribution of the future time series $P\left(\mathbf{y}_{(t+1):(t+\Omega)}|\mathbf{y}_{1:t}\right)$.

Classical generative models are often used to model time series data, which factorize the joint probability of future observations given the past information as the product of conditional probabilities:

$$P\left(\mathbf{y}_{(t+1):(t+\Omega)}|\mathbf{y}_{1:t}\right) = \prod_{\omega=1}^{\Omega} p(\mathbf{y}_{t+\omega}|\mathbf{y}_{1:t+\omega-1}), \tag{1}$$

where each future observation is conditioned on the observations at all previous timestamps. In practice, the generative models may face some challenges when applied to real-world forecasting scenarios such as demand forecasting for online retailers. In addition to the efficiency issue in both training and forecasting stages, there is also an error accumulation problem as each prediction is fed back as the ground-truth to forecast longer horizons, in which process errors

may accumulate. Instead of applying the classical generative approach, our framework forecasts the joint distribution of future observations directly:

$$P\left(\mathbf{y}_{(t+1):(t+\Omega)}|\mathbf{y}_{1:t}\right) = \prod_{\omega=1}^{\Omega} p(\mathbf{y}_{t+\omega}|\mathbf{y}_{1:t}). \tag{2}$$

While time series data usually have systematic patterns such as trend and seasonality, it is also crucial that a forecasting framework allows covariates $X_{t+\omega}^{(i)}$ (where $\omega = 1, ..., \Omega$ and $i = 1, ..., N$) that include additional information to the direct forecasting strategy in Equation 2. The joint distribution of the future incorporating the covariates becomes:

$$P\left(\mathbf{y}_{(t+1):(t+\Omega)}|\mathbf{y}_{1:t}\right) = \prod_{\omega=1}^{\Omega} p(\mathbf{y}_{t+\omega}|\mathbf{y}_{1:t}, X_{t+\omega}^{(i)}, i = 1, ..., N). \tag{3}$$

Under the above settings, the challenge becomes to design a neural network framework that incorporates the historical observations $\mathbf{y}_{1:t}$ and the covariates $X_{t+\omega}^{(i)}$. In the following sections, we describe how we extend the idea of the dynamic regression model (e.g., the ARIMAX model) to build a direct forecasting framework for multiple time series by applying dilated causal convolutions and residual neural networks. We will then describe the probabilistic forecasting framework in detail and some practical considerations of the input features.

3.1. Neural network architecture

Dynamic regression models (e.g., ARIMAX) extend the classical time series model to include both information from past observations and exogenous variables (Pankratz 2012). A way to represent dynamic regression models is as follows:

$$y_t^{(i)} = \nu_B(X_t^{(i)}) + n_t^{(i)}.$$

where $\nu_B(\cdot)$ is a transfer function that describes how the changes in exogenous variables $X_t^{(i)}$ are transferred to $y_t^{(i)}$, and $n_t^{(i)}$ is a stochastic time series process, e.g., the ARIMA process, which captures a forecast of $y_t^{(i)}$ using historical information.

To extend the dynamic regression model to multiple time series forecasting scenario, we propose a variant of residual neural network (resnet, He et al., 2016*a*,*b*). Its main difference from the original resnet is that the new block allows for two inputs – one input for the historical observations and the other for exogenous variables. for convenience, we refer it as resnet-v in the rest of the paper. Section 3.1.2 provides more details of the module resnet-v.

In this paper, we propose the Deep Temporal Convolutional Network (DeepTCN). The entire architecture of DeepTCN is presented in Figure 1a. The high-level architecture is similar to the classical Seq2Seq framework. In the encoder part, stacked dilated causal convolutions are

constructed to model the stochastic process of historical observations and output $h_t^{(i)}$. Then, the module resnet-v in the decoder part incorporates the latent output $h_t^{(i)}$ and future exogenous variables $X_{t+\omega}^{(i)}$, and outputs another latent output. Finally, a dense layer is applied to map the output of resnet-v and to produce the probabilistic forecasts of future observations. In the following sections, we provide further details for each module.

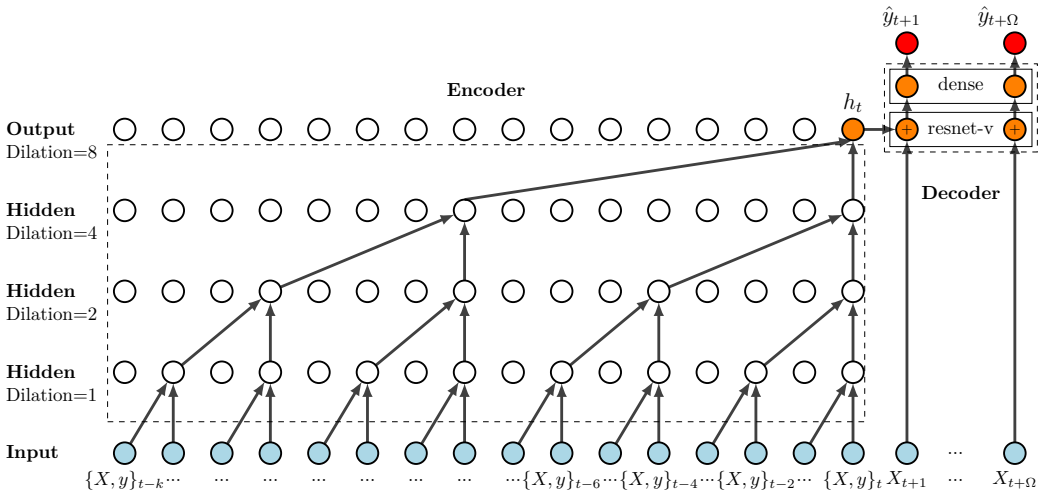
3.1.1. Encoder: Dilated causal convolutions

Causal convolutions are convolutions where the output at time t can only be obtained from the inputs that are no later than t. Dilation causal convolutions allow the filter to be applied over an area larger than its length by skipping the input values with a certain step (van den Oord et al., 2016). In the case of univariate series, given a single-dimensional input sequence x, the output (feature map) s at location t of a dilated convolution with kernel w can be expressed as:

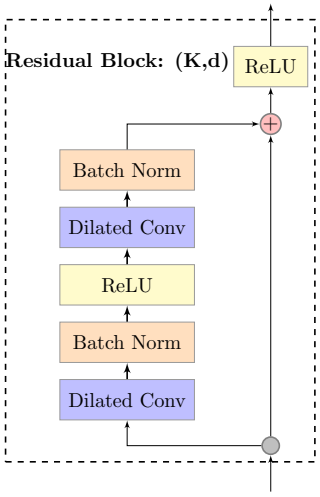
$$s(t) = (x *_d w)(t) = \sum_{k=0}^{K-1} w(k)x(t - d \cdot k), \tag{4}$$

where d is the dilation factor, and K is the size of the kernel. Stacking multiple dilated convolutions enable networks to have very large receptive fields and to capture long-range temporal dependencies with a smaller number of layers. The left part of Figure 1a is an example of dilated causal convolutions with dilation factors $d = \{1, 2, 4, 8\}$, where the filter size $K = 2$ and a receptive field of size 16 is reached by staking four layers.

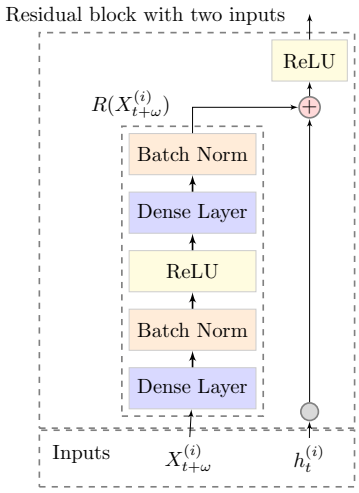
Figure 1b shows the basic module for each layer of the encoder, where both of two dilated convolutions inside the module have the same kernel size K and dilation factor d. Instead of implementing the classical gating mechanism in Wavenet (van den Oord et al., 2016), in which a dilated convolution is followed by a gating activation, residual blocks are taken as the ingredient. As shown in Figure 1b, each residual block consists of two layers of dilated causal convolutions, the first of which is followed by a batch normalization and rectified nonlinear unit (ReLU) (Nair and Hinton, 2010) while the second of which is followed by another batch normalization (Ioffe and Szegedy, 2015). The output after the second batch normalization layer is taken as the input of the residual block, followed by a second ReLU. Residual blocks have been proven to help efficiently train and stabilize the network, especially when the input sequence is very long. More importantly, non-linearity gained by the rectified linear unit (ReLU) achieves better prediction accuracy in most of the empirical studies. Various Natural Language Processing (NLP) tasks also support the above conclusion (Bai et al., 2018).



(a) Architecture of **DeepTCN**



(b) Encoder module



(c) Decoder module

Figure 1: (a) Architecture of DeepTCN. Encoder part: stacked dilated causal convolutional nets are constructed to capture the long-term temporal dependencies. Decoder part: the decoder includes a variant of residual block (referred as resnet-v, shown as $\oplus$) and an output dense layer. The module resnet-v is designed to integrate output of stochastic process of historical observations and future covariates. Then the output dense layer is adopted to map the output of resnet-v into our final forecasts. (b) Encoder module. Residual blocks are taken as the ingredient. Each residual block consists of two layers of dilated causal convolutions, the first of which is followed by a batch normalization and ReLU and the second of which is follow by another batch normalization. The output is taken as the input of the residual block, followed by another ReLU. (c) Decoder module. $h_t^{(i)}$ is the output of the encoder, $X_{t+\omega}^{(i)}$ are the future covariates, and $R(\cdot)$ is the nonlinear function applied on $X_{t+\omega}^{(i)}$. For the residual function $R(\cdot)$, we first apply a dense layer and a batch normalization to project the future covariates. Then a ReLU activation is applied followed by another dense layer and batch normalization.

3.1.2. Decoder: Residual neural network

The decoder includes two parts. The first part is the variant of residual neural network, the module resnet-v. The second part is a dense layer that maps the output of the resnet-v to the probabilistic forecasts. As mentioned before, the module resnet-v allows for two inputs (one for the historical information and the other for exogenous variables), and is designed to capture the information of these two inputs. It can be written as:

$$\delta_{t+\omega}^{(i)} = R(X_{t+\omega}^{(i)}) + h_t^{(i)},$$

where $h_t^{(i)}$ is the latent output by the encoder, $X_{t+\omega}^{(i)}$ are the future covariates and $\delta_{t+\omega}^{(i)}$ is the latent output of resnet-v. $R(\cdot)$ is the residual function applied on $X_{t+\omega}^{(i)}$. Hence the nonlinear function $R(\cdot)$ plays the role of transfer function in dynamic regression model and explains the residuals between ground truth and predictions solely determined by the encoder part (e.g, the promotion effects on online retailer platforms or weather forecast for brick and mortar retailers).

Figure 1c shows the structure of the resnet-v. For the residual function $R(\cdot)$, we first apply a dense layer and a batch normalization to project the future covariates. Then a ReLU activation is applied followed by another dense layer and batch normalization. Finally, an output dense layer maps the latent variable $\delta_{t+\omega}^{(i)}$ to produce the final output Z that corresponds to the probabilistic estimation of interest.

In the next section, we describe how we construct the probabilistic forecasting framework via neural networks in the output dense layer.

3.2. Probabilistic forecasting framework

Neural networks enjoy the flexibility to produce multiple outputs. In the DeepTCN framework, for each future observation, the output dense layer in the decoder can produce m outputs: $Z = (z^1, ..., z^m)$, which represent the parameter set of the hypothetical distribution of interest. Take Gaussian distribution as an example, for the ω-th future observation of the i-th series, $y_{t+\omega}^{(i)}$, the output layer produces two outputs (the mean and the standard deviation), which gives $Z_{t+\omega}^{(i)} = (\mu_{t+\omega}^{(i)}, \sigma_{t+\omega}^{(i)})$, where $\mu_{t+\omega}^{(i)}$ is the expectation of $y_{t+\omega}^{(i)}$ and $\sigma_{t+\omega}^{(i)}$ is the standard deviation. Therefore, the probabilistic forecasts can be described as:

$$P\left(y_{t+\omega}^{(i)}\right) \sim G(\mu_{t+\omega}^{(i)}, \sigma_{t+\omega}^{(i)}). \tag{5}$$

More specifically, we consider two probabilistic forecasting frameworks in this paper. The first one is the parametric framework, in which probabilistic forecasts of future observations can be achieved by directly predicting the parameters of the hypothetical distribution (e.g., the mean and the standard deviation for Gaussian distribution) based on maximum likelihood

estimation. The second one is non-parametric, which produces a set of forecasts corresponding to quantile points of interest (Koenker and Bassett Jr, 1978) with Z representing the quantile forecasts.

In practice, whether to choose the parametric approach or the non-parametric approach depends on the application context. The parametric approach requires the assumption of a specific probability distribution while the non-parametric approach is distribution-free and thus is usually more robust. However, a decision-making scenario may rely on the sum of probabilistic forecasts for a certain period. For example, an inventory replenishment decision may depend on the distribution of the sum of demand for the next few days. In such cases, the non-parametric approach will not work since the output (e.g., the quantiles) is not additive over time and the parametric approach has its advantage of being flexible in obtaining such information by sampling from the estimated distributions.

3.2.1. Non-parametric approach

In the non-parametric framework, forecasts can be obtained by quantile regression. In quantile regression (Koenker and Bassett Jr, 1978), denoting the observation and the prediction for a specific quantile level q as y and $\hat{y}^q$ respectively, models are trained to minimized the quantile loss, which is defined as

$$L_q(y, \hat{y}^q) = q(y - \hat{y}^q)^+ + (1 - q)(\hat{y}^q - y)^+, \tag{6}$$

where $(y)^+ = \max(0, y)$ and $q \in (0, 1)$. Given a set of quantile levels $Q = (q_1, ..., q_m)$, the m corresponding forecasts can be obtained by minimizing the total quantile loss defined as

$$L_Q = \sum_{j=1}^{m} L_{q_j}\left(y, \hat{y}^{q_j}\right).$$

3.2.2. Parametric approach

For the parametric approach, given the predetermined distribution (e.g., Gaussian distribution), the maximum likelihood estimation is applied to estimate the corresponding parameters. Take Gaussian distribution as an example, for each target value y, the network outputs the parameters of the distribution, namely the mean and the standard deviation, denoted by μ and σ, respectively. The negative log-likelihood function is then constructed as the loss function:

$$\begin{aligned} L_G &= -\log \ell(\mu, \sigma \,|y) \\ &= -\log\left((2\pi\sigma^2)^{-1/2} \exp\left[-(y-\mu)^2/(2\sigma^2)\right]\right) \\ &= \frac{1}{2}\log(2\pi) + \log(\sigma) + \frac{(y-\mu)^2}{2\sigma^2}. \end{aligned}$$

We can extend this approach to a variety of probability distribution families. For example, we can choose negative-binomial distribution for long-tail products, which is traditionally used for modeling over-dispersed count data and has been shown to perform well in empirical studies (Villani et al., 2012; Snyder et al., 2012; Syntetos et al., 2015; Salinas et al., 2019).

It is worth mentioning that some parameters of a certain distribution (e.g., the standard deviation in Gaussian distribution) must satisfy the condition of positivity. To accomplish this, we apply "Soft ReLU" activation, namely the transformation $\hat{z} = \log(1 + \exp(z))$, to ensure positivity (Salinas et al., 2019).

3.3. Input features

There are typically two kinds of input features: time-dependent features (e.g., product price and day-of-the-week) and time-independent features (e.g., product_id, product brand and category). Time-independent covariates such as product_id contain series-specific information. Including these covariates helps capture the scale level and seasonality for each specific series.

To capture seasonality, we use hour-of-the-day, day-of-the-week, day-of-the-month for hourly data, day-of-the-year for daily data and month-of-year for monthly data. Besides, we use hand-crafted holiday indicators for shopping festival such as "11.11", which enables the model to learn spikes due to scheduled events.

Dummy variables such as product_id and day-of-the-week are mapped to dense numeric vectors via embedding (Mikolov, Sutskever, Chen, Corrado and Dean, 2013; Mikolov, Chen, Corrado and Dean, 2013). We find that the model is able to learn more similar patterns across series by representation learning and thus improves the forecasting accuracy for related time series, which is especially useful for series with little or no historical data. In the case of new products or new warehouses without sufficient historical data, we perform zero padding to ensure the desired length of the input sequence.

4. Experiments

4.1. Datasets

We evaluate the performance of DeepTCN on five datasets. More specifically, within the DeepTCN framework, two models – the non-parametric model that predicts the quantiles and the parametric Gaussian likelihood model – are applied for the forecasting performance evaluation. We refer to them as `DeepTCN-Quantile` and `DeepTCN-Gaussian`, respectively, for the rest of the paper.

Table 1 shows the details of the five datasets. `JD-demand` and `JD-shipment` are from JD.com, which correspond to two forecasting tasks for online retailers, namely demand forecasting of

	JD-demand	JD-shipment	electricity	traffic	parts
Number	50,000	1,450	370	963	1,406
Length	$[0, 1800]$	$[0, 1800]$	26,304	10,560	51
Domain	$\mathbb{N}$	$\mathbb{N}$	$\mathbb{R}^+$	$[0, 1]$	$\mathbb{N}$
Granularity	daily	daily	hourly	hourly	monthly

Table 1: Summary of the datasets used in the experiments.

regional product sales and shipment forecasting of the daily delivery volume of packages in retailers' warehouses. Since it is inevitable for new products or warehouses to emerge, the training periods for these two datasets can range from zero to several years and the corresponding forecasting tasks involve situations such as cold-starts and data sparsity. We also use three public datasets that have been widely used in various time series forecasting studies for accuracy comparison, namely `electricity`, `traffic` and `parts`. The `electricity` dataset contains hourly time series of the electricity consumption of 370 customers. The `traffic` dataset is a collection of the occupancy rates (between 0 and 1) of 963 car lanes from San Francisco bay area freeways. The `parts` dataset is comprised of 1,046 time series representing monthly demand of spare parts in a US car company. A more detailed description of these datasets can be found in Appendix A.

4.2. Accuracy comparison

Current baseline models for JD.com's datasets include seasonal ARIMA (`SARIMA`) and `lightGBM`, a gradient boosting tree method that has been empirically proven to be a highly effective approach in predictive modeling. These two online models are deployed and continuously improved to provide more accurate forecasts and to better serve the consequent business operations (e.g., inventory replenishment). A more detailed description including the features and parameters used in these two models can be found in Appendix B. For the public datasets, we compare `DeepTCN` with `DeepAR` (Salinas et al., 2019) as implemented using the student-t distribution (Gasthaus et al., 2019), `SQF-RNN` (Gasthaus et al., 2019) and `DeepState` (Rangapuram et al., 2018).

4.2.1. Evaluation metrics

To evaluate probabilistic forecasting, given N time series $\{y^{(i)}\}_{i=1}^{N}$ and the prediction range $\{t+1, t+2, ..., t+\Omega\}$, we use the normalized sum of quantile losses (Gasthaus et al., 2019),

which is denoted as

$$QL_q = \frac{\sum_{i,t} L_q(y_t^{(i)}, \hat{y}_t^{(i)})}{\sum_{i,t} |y_t^{(i)}|},$$

where $L_q(\cdot)$ is defined in Equation 6. We refer to QL_q as the q-quantile loss.

The evaluation metrics used in our experiments for point forecasting include the Symmetric Mean Absolute Percent Error (SMAPE), the Normalized Root Mean Square Error (NRMSE) and the Mean Absolute Scaled Error (MASE), which are defined as follows. Note that for the MASE, the value m is the seasonal frequency.

$$\begin{aligned}
SMAPE &= \frac{1}{N(\Omega - t)} \sum_{i,t} \left| \frac{2\left(y_t^{(i)} - \hat{y}_t^{(i)}\right)}{y_t^{(i)} + \hat{y}_t^{(i)}} \right|, \\
NRMSE &= \frac{\sqrt{\frac{1}{N(\Omega-t)} \sum_{i,t} \left(y_t^{(i)} - \hat{y}_t^{(i)}\right)^2}}{\frac{1}{N(\Omega-t)} \sum_{i,t} \left|y_t^{(i)}\right|}, \\
MASE &= \frac{1}{N(\Omega - t)} \sum_{i} \frac{\sum_{t} \left|y_t^{(i)} - \hat{y}_t^{(i)}\right|}{\frac{1}{T-m} \sum_{t=m+1}^{T} \left|y_t^{(i)} - y_{t-m}^{(i)}\right|},
\end{aligned}$$

where $y_t^{(i)}$ is the true value of series i at time step t, and $\hat{y}_t^{(i)}$ is the corresponding prediction value.

4.2.2. Results on JD.com's datasets

We start with comparing the probabilistic forecasting results of `DeepTCN` against the online `SARIMA` and `lightGBM` models on JD.com datasets over two testing periods: Oct 2018 and Nov 2018. In particular, China's largest shopping festival "11.11" lasts from Nov 1 to Nov 12, during which Nov 11 is the biggest promotion day. We use the 0.5-quantile loss and 0.9-quantile loss as the evaluation metrics, which are referred to as QL50 and QL90, respectively. The model `DeepTCN-Quantile` is trained to predict q-quantiles with $q \in \{0.5, 0.9\}$. For the model `DeepTCN-Gaussian`, the quantile predictions are obtained by calculating the percent point function of Gaussian distribution (the inverse of cumulative density function) at 0.5 and 0.9 quantiles.

The comparison results of `JD-demand` and `JD-shipment` are illustrated in Table 2. As we can see, both `DeepTCN-Quantile` and `DeepTCN-Gaussian` achieve better results than the two online models. In particular, `DeepTCN-Quantile` performs the best. One possible reason is that `DeepTCN-Gaussian` is constructed based on the Gaussian likelihood, but these datasets do not necessarily follow the assumption of normal distribution. On the contrary, the model

Method	JD-demand		JD-shipment	
	Oct 2018	Nov 2018	Oct 2018	Nov 2018
SARIMA	0.377/0.313	0.385/0.482	0.154/0.098	0.199/0.158
lightGBM	0.366/0.303	0.392/0.494	0.144/0.087	0.205/0.136
DeepTCN-Quantile	**0.328/0.264**	**0.349/0.350**	**0.087/0.050**	**0.124/0.080**
DeepTCN-Gaussian	0.349/0.294	0.360/0.436	0.094/0.052	0.163/0.109

Table 2: Comparison of probabilistic forecasts on JD-demand and JD-shipment datasets. The quantile losses QL50/QL90 are evaluated against online models over two testing periods – Oct 2018 and Nov 2018.

DeepTCN-Quantile, in light of the distribution-free nature, generates better forecasts by minimizing the quantile loss directly.

We then present in Table 3 an accuracy comparison of point forecasting between our model and the other two baseline models including SARIMA and lightGBM. The point forecasting results of DeepTCN-Quantile are achieved with the non-parametric approach that predicts the 0.5 quantiles. In Table 3, All-Data consists of all series in the dataset; Long-series includes series with historical data longer than two years; Short-series are those starting after 2018, i.e., without historical shopping festival data. We can see that DeepTCN-Quantile achieves consistently the best accuracy with regard to all the metrics across all data groups. In particular, when historical shopping festival data is not available, the performance of SARIMA and lightGBM become much worse (the result in Short-series), while DeepTCN-Quantile maintains the same performance level.

To gain a better understanding of the performance improvement exhibited by the proposed DeepTCN framework, we show in Figure 2 three cases of probabilistic forecasts generated by SARIMA and DeepTCN-Quantile. Case A and Case B are two demand forecasting examples of Oct 2018 and Nov 2018, respectively, while Case C is an example of shipment forecasting of Nov 2018. It is shown that for both tasks, DeepTCN-Quantile generates more accurate uncertainty estimation. Moreover, SARIMA postulates increasing uncertainty over time while the uncertainty estimation of DeepTCN-Quantile is well learned from the data. For example, the uncertainty of SARIMA during the shopping festival period is huge due to both promotion activities and intense market competition.

Finally, we perform a qualitative analysis on JD-shipment dataset over the testing period of Nov 2018. We choose this dataset because 1) it consists of series whose magnitudes of volume are high and stable, and 2) the testing period involves China's biggest shopping festival "11.11". As mentioned before, the occurrence of this festival results in a spike for the shipment volume, in which the forecasting tasks become more challenging. In Figure 3, we illustrate

Data group	Method	NRMSE	SMAPE	MASE
	`SARIMA`	1.285	0.369	0.911
`All-data`	`lightGBM`	1.244	0.430	0.918
	`DeepTCN-Quantile`	**0.873**	**0.284**	**0.763**
	`SARIMA`	1.044	0.323	0.900
`Long-series`	`lightGBM`	0.995	0.312	0.903
	`DeepTCN-Quantile`	**0.895**	**0.268**	**0.708**
	`SARIMA`	1.376	0.430	0.980
`Short-series`	`lightGBM`	1.536	0.457	1.096
	`DeepTCN-Quantile`	**0.923**	**0.354**	**0.798**

Table 3: Point forecasting accuracy comparison on NRMSE, SMAPE and MASE of different subgroups of `JD-shipment` in Nov 2018. `All-Data` represents all series with the length of training periods ranging from zero to four years; `Long-series` includes the warehouses with historical data of more than two years; `Short-series` indicates series starting after 2018, namely those with no historical shopping festival data.

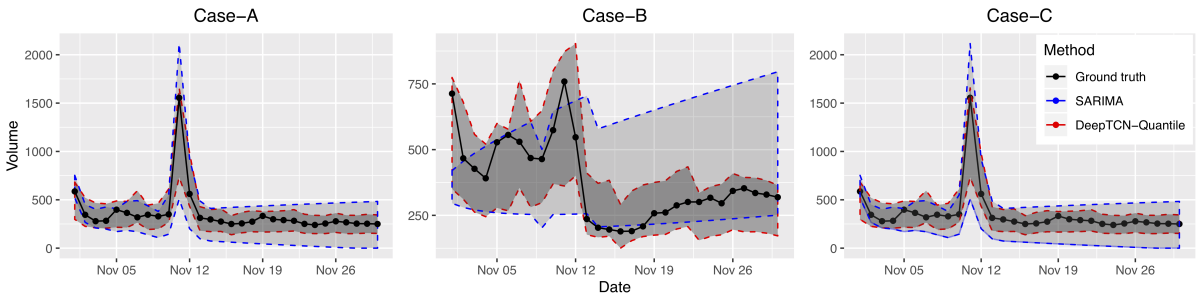


Figure 2: Probabilistic forecasts of `SARIMA` and `DeepTCN-Quantile` for three cases (randomly chosen for illustration purposes). Case A and Case B show the forecasting results of two fast-moving products; Case C shows the forecasting results of the daily delivery volume of packages from one warehouse. The ground truth, and the [10%, 90%] prediction intervals of `SARIMA` and `DeepTCN-Quantile` are also shown in different colors. (For interpretation of the references to colour in this figure, the reader is referred to the web version of this article.)

cases of point forecasting under three different scenarios. "11.11" is the major promotion day and we can observe a spike in the true volume. In Cases A-1 and A-2 , where historical data of more than two years is available, all models can learn a similar volume pattern, including the spike on "11.11". However, `SARIMA` and `lightGBM` in Cases B-1 and B-2 fail to capture the spike on "11.11" due to lack of sufficient training data for historical festivals. Cases C-1 and C-2 are selected to demonstrate how these models handle cold-start forecasting. It turns out that `DeepTCN-Quantile` stands out for this situation as it is able to capture both scale and shape patterns of the new warehouses by learning data from other warehouses with similar store-specific features.

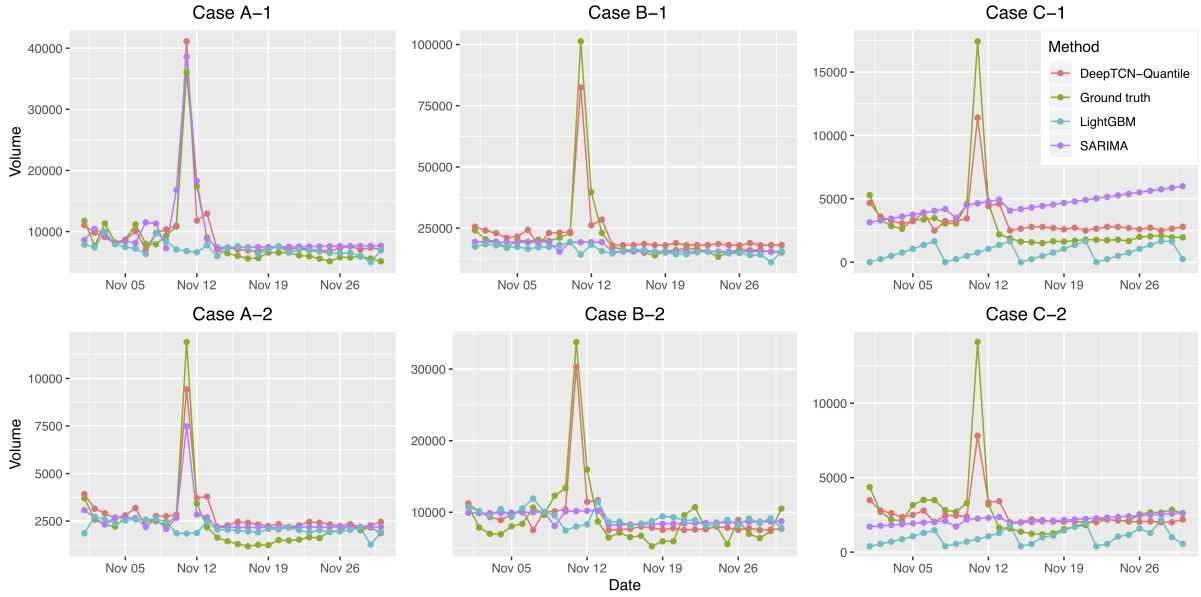


Figure 3: Point forecasts of `DeepTCN-Quantile`, `SARIMA` and `lightGBM` for six cases (randomly chosen from `JD-shipment` for illustration purposes). Cases A-1 and A-2 are examples with historical data of more than two years; cases B-1 and B-2 show instances without previous shopping festival data; cases C-1 and C-2 illustrate cold-start forecasting namely the forecasting of time series with little historical data, e.g., less than three days. Note that Nov 11 is one of China's biggest promotion days. (For interpretation of the references to colour in this figure, the reader is referred to the web version of this article.)

4.2.3. Results on the public datasets

Our first experiment on public datasets is to evaluate the performance of DeepTCN regarding probabilistic forecasting. For `electricity` and `traffic` dataset, we implement a 24-hour ahead forecasting task for last seven days based on a rolling-window approach as described in Salinas et al. (2019). For `parts` dataset, we evaluate the performance for last 12 months. It is worth noting that we use the same model trained on the data before the first prediction window rather than retraining the model after updating the forecasts. In all forecasting experiments,

we train the DeepTCN-Quantile models to predict q-quantiles with $q \in \{0.5, 0.9\}$, and for the model DeepTCN-Gaussian, the quantile predictions are obtained by calculating the percent point function of Gaussian distribution at 0.5 and 0.9 quantiles, which is the same approach as applied in JD.com's datasets.

Dataset	ETS	auto.arima	DeepAR-t	SQF-RNN	DeepState	DeepTCN-Quantile	DeepTCN-Gaussian
electricity	0.100/0.050	0.142/0.054	0.068/0.033	0.066/0.035	**0.043/0.025**	0.057/0.029	0.062/0.039
traffic	0.360/0.325	0.246/0.140	0.117/0.090	0.119/0.090	0.084/0.058	**0.058/0.040**	**0.071/0.049**
parts	0.820/0.505	0.822/0.533	0.636/0.543	—/—	0.735/0.467	**0.533/0.462**	**0.623/0.465**

Table 4: Accuracy comparison of probabilistic forecasting on public datasets. The numbers in the table are the QL50/QL90 results.

Table 4 illustrates the probabilistic forecasting results obtained by these models. It can be seen that the probabilistic forecasting results of both DeepTCN-Quantile and DeepTCN-Gaussian outperform other state-of-the-art models on traffic and parts datasets. One possible reason is that there exists high correlation within these two datasets, and DeepTCN takes an advantage by learning the non-linear correlation among series, while traditional models such as ETS and auto.arima could not learn the shared patterns across the time series.

Dataset	Method	NRMSE	SMAPE	MASE
electricity	ETS	0.838	0.156	1.247
	DeepAR-t	0.550	**0.110**	0.931
	SQF-RNN	**0.518**	0.113	0.937
	DeepTCN-Quantile	0.523	0.118	**0.926**
traffic	ETS	0.872	0.594	1.881
	DeepAR-t	0.396	**0.104**	0.442
	SQF-RNN	0.381	0.117	0.449
	DeepTCN-Quantile	**0.364**	0.110	**0.438**

Table 5: Accuracy comparison of point forecasting on public datasets.

To evaluate the performance of point forecasting, we comparing DeepTCN (DeepTCN that predicts the 0.5 quantiles) against ETS, DeepAR-t (Salinas et al., 2019) and SQF-RNN (Gasthaus et al., 2019) and calculate the metrics NRMSE, SMAPE and MASE over the electricity and traffic datasets. As can be seen from Table 5, for traffic dataset with highly correlated series, DeepTCN-Quantile achieves the best forecasting accuracy on NRMSE and MASE, while

it performs the best on MASE for `electricity`.

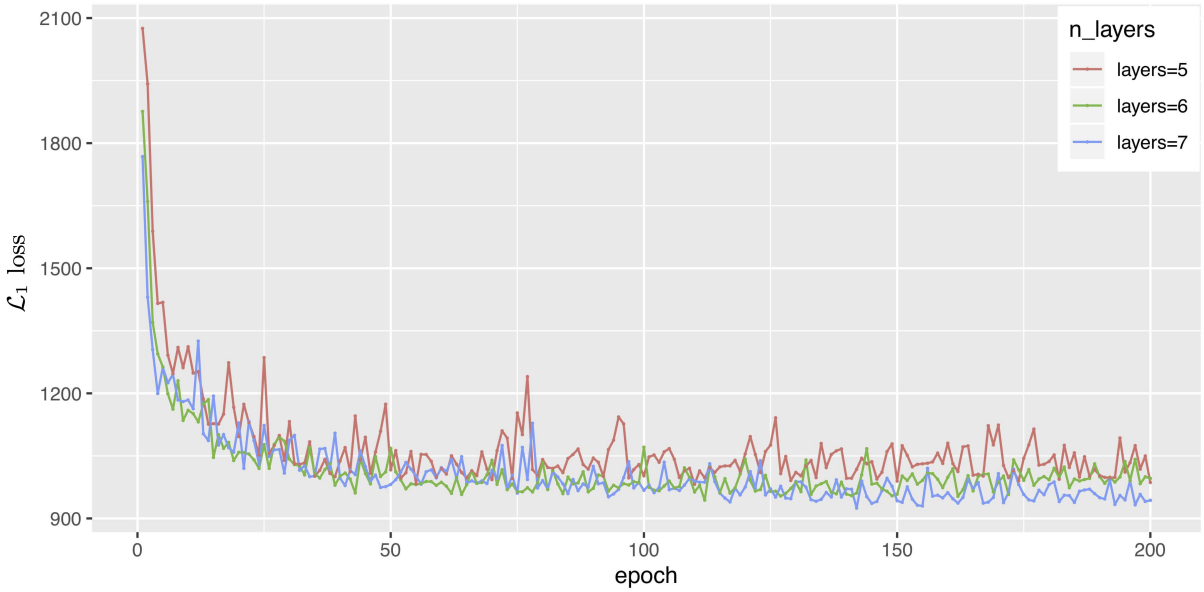


Figure 4: $\mathcal{L}_1$ loss over 200 epochs of three different architectures with 5, 6 and 7 encoder layers.

4.3. Sensitivity analysis

Given that the stochastic process of historical observations is modeled by the stacked dilated causal convolutions in the encoder part of our DeepTCN framework, we now perform sensitivity analysis taking the `traffic` dataset as an example, to explore the effect of the number of encoder layers on the model performance. In this experiment, we set the filter size of the dilated causal convolutions as $k = 2$ and implement three model architectures: (1) a 5-layer architecture with dilation factors $d = \{1, 2, 4, 8, 16\}$, (2) a 6-layer architecture with dilation factors $d = \{1, 2, 4, 8, 16, 32\}$ and (3) a 7-layer architecture with dilation factors $d = \{1, 2, 4, 8, 16, 20, 32\}$. Notice that in our experiment of traffic dataset, the length of input sequence is $7 \times 24 = 168$, which is the hourly data of the previous week. And for each layer of the dilated causal convolutions, the kernel size times the dilation factor cannot exceed the length of the input sequence, thus we set the dilation factor of the third model as $d = \{1, 2, 4, 8, 16, 20, 32\}$ to ensure the input length of each layer is sufficient.

Figure 4 shows the $\mathcal{L}_1$ loss of these three models over 200 epochs. It can be seen that both the 6-layer and 7-layer architectures perform better than the 5-layer architecture. One reason is that the 5-layer architecture is relatively shallow to fully model the information from the historical observations. However, as one can see, the difference between using 6-layer and 7-layers is small, meaning that as long as one uses enough number of layers, the difference in result is quite small. Such phenomenon is quite consistent across other test cases. Thus our method is quite robust with respect to the model parameters.

5. Conclusion

We present a convolutional-based probabilistic forecasting framework for multiple related time series and show both non-parametric and parametric approaches to model the probabilistic distribution based on neural networks. Our solution can help in the design of practical large-scale forecasting applications, which involves situations such as cold-starts and data sparsity. Results from both industrial datasets and public datasets show that the framework yields superior performance compared to other state-of-the-art methods in both point and probabilistic forecasting.

Acknowledgements

Yanfei Kang's research were supported by the National Natural Science Foundation of China (No. 11701022).

References

References

Bai, S., Kolter, J. Z. and Koltun, V. (2018), 'An empirical evaluation of generic convolutional and recurrent networks for sequence modeling', *arXiv preprint arXiv:1803.01271* .

Bandara, K., Bergmeir, C. and Smyl, S. (2020), 'Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach', *Expert Systems With Applications* **140**, 112896.

Bianchi, F. M., Maiorino, E., Kampffmeyer, M. C., Rizzi, A. and Jenssen, R. (2017), 'An overview and comparative analysis of recurrent neural networks for short term load forecasting', *arXiv preprint arXiv:1705.04378* .

Borovykh, A., Bohte, S. and Oosterlee, C. W. (2017), 'Conditional time series forecasting with convolutional neural networks'. arXiv preprint arXiv:1703.04691.

Box, G. E., Jenkins, G. M., Reinsel, G. C. and Ljung, G. M. (2015), *Time Series Analysis: Forecasting and Control*, John Wiley & Sons.

Chen, T., Li, M., Li, Y., Lin, M., Wang, N., Wang, M., Xiao, T., Xu, B., Zhang, C. and Zhang, Z. (2015), 'Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems', *arXiv preprint arXiv:1512.01274* .

Cho, K., Van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014), Learning phrase representations using RNN encoder-decoder for statistical machine translation, *in* 'Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)', pp. 1724–1734.

Gasthaus, J., Benidis, K., Wang, Y., Rangapuram, S. S., Salinas, D., Flunkert, V. and Januschowski, T. (2019), Probabilistic forecasting with spline quantile function RNNs, *in* 'The 22nd International Conference on Artificial Intelligence and Statistics', pp. 1901–1910.

Graves, A. (2013), 'Generating sequences with recurrent neural networks', *arXiv preprint arXiv:1308.0850* .

Gu, J., Bradbury, J., Xiong, C., Li, V. O. and Socher, R. (2017), 'Non-autoregressive neural machine translation', *arXiv preprint arXiv:1711.02281* .

He, K., Zhang, X., Ren, S. and Sun, J. (2016*a*), Deep residual learning for image recognition, *in* 'Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition', pp. 770–778.

He, K., Zhang, X., Ren, S. and Sun, J. (2016*b*), Identity mappings in deep residual networks, *in* 'European Conference on Computer Vision', Springer, pp. 630–645.

Hyndman, R. J. and Athanasopoulos, G. (2018), *Forecasting: Principles and Practice*, OTexts.

Hyndman, R., Koehler, A. B., Ord, J. K. and Snyder, R. D. (2008), *Forecasting with Exponential Smoothing: The State Space Approach*, Springer Science & Business Media.

Ioffe, S. and Szegedy, C. (2015), Batch normalization: Accelerating deep network training by reducing internal covariate shift, *in* 'International Conference on Machine Learning', pp. 448–456.

Kaggle (2017), 'Web traffic time series forecasting', `https://www.kaggle.com/c/web-traffic-time-series-forecasting`.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017), LightGBM: A highly efficient gradient boosting decision tree, *in* 'Advances in Neural Information Processing Systems', pp. 3146–3154.

Koenker, R. and Bassett Jr, G. (1978), 'Regression quantiles', *Econometrica* **46**(1), 33–50.

Laptev, N., Yosinski, J., Li, L. E. and Smyl, S. (2017), Time-series extreme event forecasting with neural networks at Uber, *in* 'International Conference on Machine Learning'.

Lv, Y., Duan, Y., Kang, W., Li, Z. and Wang, F.-Y. (2015), 'Traffic flow prediction with big data: A deep learning approach', *IEEE Transactions on Intelligent Transportation Systems* **16**(2), 865–873.

Maddix, D. C., Wang, Y. and Smola, A. (2018), 'Deep factors with Gaussian processes for forecasting', *arXiv preprint arXiv:1812.00098* .

Makridakis, S., Spiliotis, E. and Assimakopoulos, V. (2018*a*), 'The M4 competition: Results, findings, conclusion and way forward', *International Journal of Forecasting* **34**(4), 802–808.

Makridakis, S., Spiliotis, E. and Assimakopoulos, V. (2018*b*), 'Statistical and machine learning forecasting methods: Concerns and ways forward', *PloS one* **13**(3), e0194889.

Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013), 'Efficient estimation of word representations in vector space', *arXiv preprint arXiv:1301.3781* .

Mikolov, T., Karafiát, M., Burget, L., Černockỳ, J. and Khudanpur, S. (2010), Recurrent neural network based language model, *in* 'Eleventh Annual Conference of the International Speech Communication Association', pp. 1045–1048.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S. and Dean, J. (2013), Distributed representations of words and phrases and their compositionality, *in* 'Advances in Neural Information Processing Systems', pp. 3111–3119.

Nair, V. and Hinton, G. E. (2010), Rectified linear units improve restricted boltzmann machines, *in* 'Proceedings of the 27th International Conference on Machine Learning', pp. 807–814.

Pankratz, A. (2012), *Forecasting with Dynamic Regression Models*, Vol. 935, John Wiley & Sons.

Pascanu, R., Mikolov, T. and Bengio, Y. (2013), On the difficulty of training recurrent neural networks, *in* 'International Conference on Machine Learning', pp. 1310–1318.

Rangapuram, S. S., Seeger, M. W., Gasthaus, J., Stella, L., Wang, Y. and Januschowski, T. (2018), Deep state space models for time series forecasting, *in* 'Advances in Neural Information Processing Systems', pp. 7795–7804.

Sagheer, A. and Kotb, M. (2019), 'Time series forecasting of petroleum production using deep lstm recurrent networks', *Neurocomputing* **323**, 203 – 213.

Salinas, D., Flunkert, V., Gasthaus, J. and Januschowski, T. (2019), 'DeepAR: Probabilistic forecasting with autoregressive recurrent networks', *International Journal of Forecasting* . Forthcoming.

Shen, Z., Zhang, Y., Lu, J., Xu, J. and Xiao, G. (2019), 'A novel time series forecasting model with deep learning', *Neurocomputing* . Forthcoming.

Smith, T. G. (2017), 'pmdarima: ARIMA estimators for python', `https://www.alkaline-ml.com/pmdarima/`.

Smyl, S. (2016), 'Forecasting short time series with LSTM neural networks', `https://gallery.azure.ai/Tutorial/Forecasting-Short-Time-Series-with-LSTM-Neural-Networks-2`.

Snyder, R. D., Ord, J. K. and Beaumont, A. (2012), 'Forecasting the intermittent demand for slow-moving inventories: A modelling approach', *International Journal of Forecasting* **28**(2), 485 – 496.

Stepnicka, M. and Burda, M. (2016), Computational intelligence in forecasting (cif) 2016 time series forecasting competition, *in* 'IEEE WCCI 2016, IJCNN-13 Advances in Computational Intelligence for Applied Time Series Forecasting (ACIATSF)', IEEE.

Suilin, A. (2017), '1st place solution of kaggle web traffic time series forecasting', `https://github.com/Arturus/kaggle-web-traffic`.

Sutskever, I., Vinyals, O. and Le, Q. V. (2014), Sequence to sequence learning with neural networks, *in* 'Advances in Neural Information Processing Systems', pp. 3104–3112.

Syntetos, A. A., Babai, M. Z. and Gardner, E. S. (2015), 'Forecasting intermittent inventory demands: Simple parametric methods vs. bootstrapping', *Journal of Business Research* **68**(8), 1746 – 1752.

van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. W. and Kavukcuoglu, K. (2016), 'Wavenet: A generative model for raw audio', *arXiv preprint arXiv:1609.03499* .

Villani, M., Kohn, R. and Nott, D. J. (2012), 'Generalized smooth finite mixtures', *Journal of Econometrics* **171**(2), 121 – 133.

Wen, R., Torkkola, K. and Narayanaswamy, B. (2017), 'A multi-horizon quantile recurrent forecaster', *arXiv preprint arXiv:1711.11053* .

Werbos, P. J. (1990), 'Backpropagation through time: What it does and how to do it', *Proceedings of the IEEE* **78**(10), 1550–1560.

Yu, H.-F., Rao, N. and Dhillon, I. S. (2016), Temporal regularized matrix factorization for high-dimensional time series prediction, *in* 'Advances in neural information processing systems', pp. 847–855.

Appendices

A. Dataset

1. `JD-demand`. The `JD-demand` dataset is a collection of 50,000 time series of regional demand which involves around 6,000 products of `3C` (short for communication, computer and consumer electronics) category from seven regions of China. The dataset is gathered from 2014-01-01 to 2018-12-01. The features set for `JD-demand` includes historical demand and the product-specific information (e.g., region_id, product categories, brand, the corresponding product price and promotions).

2. `JD-shipment`. The `JD-shipment` dataset includes about 1450 time series from 2014-10-01 to 2018-12-01, including new series (warehouses) that emerge with the development of the companies' business. The covariates consist of historical demand, the warehouse specific info including geographic and metropolitan informations (e.g., geo_region, city) and warehouse categories (e.g. food, fashion, appliances).

3. `Electricity`. The `electricity` dataset describes the series of the electricity consumption of 370 customers, which can be accessed at `https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014`, The electricity usage values are recorded per 15 minutes from 2011 to 2014. We select the data of the last three years. By aggregating the records of the same hour, we use the hourly consumption data of size $N \times T = 370 \times 26304$, where N is the number of time series and T is the length (Yu et al., 2016).

4. `Traffic`. The `traffic` dataset describes the occupancy rates (between 0 and 1) of 963 car lanes from San Francisco bay area freeways, which can be accessed at `https://archive.ics.uci.edu/ml/datasets/PEMS-SF` The measurements are carried out over the period from 2008-01-01 to 2009-03-30 and are sampled every 10 minutes. The original dataset was split into training and test parts, and the daily order was shuffled. The total datasets were merged and rearranged to make sure it followed the calendar order. Hourly aggregation was applied to obtain hourly traffic data (Yu et al., 2016). Finally, we get the dataset of size $N \times T = 963 \times 10560$, with the occupancy rates at each station described by a time series of length $10,560$.

5. `Parts`. The `parts` dataset includes 2,674 time series supplied by a US car company, which represents the monthly sales for slow-moving parts and covers a period of 51 months. The data can be accessed at `http://www.exponentialsmoothing.net/supplements#data`. After applying two filtering rules as follows:

   - Removing series possessing fewer than ten positive monthly demands.
   - Removing series having no positive demand in the first 15 and final 15 months.

   There are finally 1,046 time series left and a more detailed description can be find in Hyndman et al. (2008).

B. Baselines

Forecasting in industrial applications often relies on a combination of univariate forecasting models and machine-learning based methods.

1. `SARIMA`: Seasonal ARIMA (`SARIMA`) is a widely used time series forecasting model which extends the ARIMA model by including additional seasonal term and is capable of modeling seasonal behaviors from the data (Box et al., 2015). Currently, `SARIMA` is applied to `JD-shipment` dataset and fast-moving products with historical data of length more than 14 in `JD-demand` dataset. The model is implemented with Python's package `pmdarima` (Smith, 2017) and the best parameters are automatically select based on the criterion of minimizing the AICs (Hyndman and Athanasopoulos, 2018). The predictions at confidence level {10%, 90%} are taken as the probabilistic forecasts in our experiments.

2. `lightGBM`: Gradient boosting tree method has been empirically proven to be a highly effective approach in predictive modeling. As one of efficient implementation of the gradient boosting tree algorithm, `lightGBM` has gained popularity of being the winning algorithm in numerous machine learning competitions, like Kaggle Competition (Ke et al., 2017). `lightGBM` is applied to both `JD-demand` dataset and `JD-shipment` dataset. The features for forecasting on `JD-shipment` are presented in Table 6. A grid-search is used to find the best values of parameters like learning rate, the depth-of-tree based on the offline evaluation on data from both last month and the same month of last year.

C. Experiment details

The current model is implemented with `Mxnet` (Chen et al., 2015) and its new high-level interface Gluon. We trained our model on a `GPU` server with one `Tesla P40` and 16 CPU (3.4

GHz). Multiple-GPU can be applied to speed up and achieve better training efficiency in real industrial application. The code for public datasets is available from https://github.com/oneday88/deepTCN.

For the JD.com's datasets, the training range and prediction horizon are both 31 days. We implement two models for both JD-demand and JD-shipment datasets. One model is trained on the data before Oct 2018 and produces forecasting on Oct 2018; the other one is trained on the data before Nov 2018 and produces forecasting on Nov 2018.

For the parts dataset, we use the first 39 months as training data and the last 12 months for evaluation. A rolling window approach with window size =4 is adopted. The training and prediction range are both 12 months and a rolling window approach with window size 4 is adopted. For both electricity and traffic datasets, the training range and prediction range

Table 6: lightGBM feature lists

Feature type	Details
Category	region_id, city_id, warehouse_type, holiday_indicators, is-weekend, etc.
Stats of warehouse level	summary (mean,median) of last week and last two weeks, summary (median, standard deviation) of last four weeks, etc .
Stats of city level	summary (mean,median) of last week and last two weeks, summary (median, standard deviation) of last four weeks, etc .
Stats of warehouse-type level	summary (mean,median) of last week and last two weeks, summary (median, standard deviation) of last four weeks, etc.

Table 7: Dataset details and deepTCN parameters

	JD-demand	JD-shipment	electricity-quantile	traffic	parts
number of time series	50,000	1,450	370	963	1,406
input-output length	31-31	31-31	168-24	168-24	12-12
dilation-list	[1,2,4,8]	[1,2,4,8]	[1,2,4,8,16,20,32]	[1,2,4,8,16,20,32]	[1,2]
number of hidden layers	6	6	9	9	4
number of training samples	200k	40k	30k	26k	4k
batch size	16	512	512	128	8
learning rate	1e-2	5e-2	5e-2	1e-2	1e-4

are selected as 168 hours and 24 hours respectively. For `electricity` dataset, we use only samples taken in December of 2011, 2012 and 2013 as training data, as we assume that this small data set is sufficient for the task of forecasting electricity consumption during the last seven days of December 2014. For `traffic` dataset, we train models on all the data before last seven days.

For each dataset, we fit the model on the training data and evaluate the corresponding metrics on the testing data after every epoch. When the training process is complete, we pick the model that gains the best evaluation results on the test set.

Convolution-related hyper-parameters, such as kernel size, number of channels and dilation length, are selected according to different tasks and datasets. The most important principle for choosing kernel size and dilation length is to make sure that the encoder (stacked residual blocks) has sufficiently large receptive field, namely long effective history of the time series. The number of channels at each convolution layer is determined by the number of input features and is kept fixed for all residual blocks. We manually tune for each dataset training-related hyper-parameters, including batch size and learning rate, in order to achieve the best performance on both evaluation metrics and running time. A more detailed description of parameters is presented in Table 7.